Jedha
AI SCHOOL

Gestionnaire de capteurs : nettoyage, structuration et tri

Fondements Mathématiques, Algorithmique et Python — projet noté (TP2) — rendu vendredi 16 octobre 2026, 14h

---

MODALITÉS

- **Durée** : 3 heures, en classe.
- **Travail en binôme** (2 étudiants par groupe).
- **Outils obligatoires** : Python 3.10+, VS Code, **Git et un compte GitHub** (`git init` , `git add` , `git commit -m "..."` vus en Phase 1 du cours).
- **IA interdite** pendant la séance, comme pour les TP précédents du module.
- **Rendu** : lien vers votre dépôt GitHub (ou, à défaut de push possible, une archive `.zip` incluant le dossier `.git`). Pas de rapport écrit séparé attendu : vos docstrings et vos messages de commit documentent vos choix.
- **Date limite** : vendredi 16 octobre 2026, 14h.
- Il n'y a pas une seule bonne façon de structurer le code : organisez votre projet comme vous le souhaitez, du moment que les exigences ci-dessous sont respectées.

Contexte

En conditions réelles, un système surveille rarement un capteur isolé : il gère un **réseau de capteurs hétérogènes**, dont il faut nettoyer, structurer et comparer les données.

Les mesures brutes de 4 capteurs vous sont fournies dans `mesures_capteurs_reseau.json` : `temperature_bureau` , `humidite_serre` , `pression_capteur_ext` , `luminosite_atelier` . Chacun contient des valeurs manquantes, non numériques et aberrantes, réparties sur plusieurs capteurs à la fois.

> **Objectif : construire un gestionnaire capable de nettoyer, structurer et comparer les mesures de plusieurs capteurs, en justifiant par une analyse de complexité la méthode choisie pour les trier.**

Exigences techniques (reprises du cadrage du cours, Phase 5.2)

Votre projet doit combiner :

1. **Une classe** `Capteur` (ou équivalent) qui encapsule le nom d'un capteur, ses mesures brutes, ses mesures nettoyées, et expose au minimum : le nombre de valeurs valides, la moyenne, le minimum, le maximum.
2. **Une structure de données** pour regrouper les 4 capteurs (dictionnaire nom → objet `Capteur` , ou liste — à vous de choisir et de justifier brièvement en commentaire).
3. **Un système de tri** pour classer les 4 capteurs selon un critère de votre choix (nombre d'anomalies, moyenne, amplitude...).

Vous êtes libres de la méthode de nettoyage et du critère de tri retenu.

Jalons

Jalon 1 — Code et dépôt Git

VENDREDI 16/10/2026, 14H

Description détaillée

- Initialisez un dépôt Git (`git init`) et créez la structure de fichiers.
- Chargez et nettoyez les données des 4 capteurs, encapsulées dans une classe `Capteur`.
- Choisissez une structure de données pour regrouper les capteurs.
- Implémentez le tri des capteurs selon le critère choisi, avec l'analyse de complexité en commentaire.
- Committez régulièrement pour tracer votre progression : **au moins 4 commits** (pas besoin de branches pour ce projet). Poussez sur GitHub avant l'échéance.

Livrables détaillés

- Lien vers le dépôt GitHub (ou `.zip` avec `.git` inclus)
- Fichier(s) `.py` avec des commentaires sur la classe et les fonctions clés
- Le fichier `mesures_capteurs_reseau.json` fourni (inchangé)

Jalon 2 — Restitution orale

VENDREDI 16/10/2026, MÊME SÉANCE

Description détaillée

- Présentation en binôme, **10 minutes**, les deux étudiants doivent intervenir :
- 1. Problème choisi et choix de structure/algorithme (2 min)
- 2. Démonstration du code en fonctionnement (4 min)
- 3. Analyse de complexité et justification des choix (2 min)
- 4. Difficultés rencontrées et ce qui a été appris (2 min)

Livrables détaillés

- Présentation orale — aucun support de diapositives imposé, une démonstration du code en direct est acceptée.

Objectifs pédagogiques

Savoirs

- Identifier une structure de données appropriée pour regrouper des objets similaires (dictionnaire vs liste)
- Définir et comparer la complexité de deux algorithmes de tri (notation Big O)

Savoir-faire

- Concevoir une classe qui encapsule des données et leur nettoyage
- Implémenter un algorithme de tri vu en cours sur une collection d'objets
- Utiliser Git pour tracer la progression d'un travail en binôme (commits réguliers, push)

Savoir-être

- Collaborer efficacement en binôme sous contrainte de temps (3h)
- Présenter et défendre ses choix techniques à l'oral

Roadmap

Jalon	Livrables attendus	Date	Moyens / formats
1	Dépôt Git (code : nettoyage, classe `Capteur` , structure, tri + complexité)	Vendredi 16/10/2026, 14h	Lien GitHub ou `.zip` avec `.git`
2	Restitution orale (10 min, en binôme)	Vendredi 16/10/2026, même séance	Présentation orale devant le formateur

Modalités de soutenance

- **Durée** : 10 minutes par binôme, structure imposée en 4 temps (voir Jalon 2).
- **Format** : présentation devant le formateur, démonstration du code en direct acceptée, pas de diapositives imposées.
- **Composition** : les deux membres du binôme doivent intervenir.
- **Critères d'évaluation** : qualité et fonctionnalité du nettoyage, pertinence de la structure et de la classe, justesse de l'algorithme de tri et de son analyse de complexité, régularité des commits Git, clarté de la restitution orale.